

STUDENT COMMAND INTERPRETER

OSSP Project – Review 1

ABSTRACT

- **Student Command Interpreter customized is a Linux shell developed using C programming.**
- It allows users to execute Linux commands through a command-line interface.
- It provides additional student-related utilities in the same shell.
- It simplifies access to common system and student-related operations.
- The project demonstrates important Operating System concepts.
- It uses system calls such as fork(), exec(), and wait() for process management.
- The main aim is to create a simple, efficient, and student-friendly command interpreter.

PROBLEM STATEMENT

The standard Linux shell is mainly designed to perform general system-level operations. It does not provide utilities specifically designed for students.

*Problems in the existing approach:

- Students need to use different commands or applications for different tasks.
- Linux commands can be difficult for beginners to understand.
- There are no built-in student-specific utilities.
- Academic-related tasks are not integrated into the shell.
- There is a need for a simple and customized student-friendly shell.

*Problem Identified

Lack of a student-focused command-line environment that combines Linux operations with useful student utilities.

PROPOSED SYSTEM

The proposed system is a custom Linux shell developed using C programming. It combines normal Linux command execution with additional student-related utilities.

Key Features

- Accepts commands from the user through a command-line interface.
- Parses and identifies the entered command.
- Executes standard Linux commands such as ls, pwd, cd, and mkdir.
- Provides student-related utilities such as student information and attendance.
- Creates and manages processes using fork().
- Executes commands using exec().
- Uses wait() for process synchronization.
- Displays appropriate output and error messages

DRAWBACKS IN EXISTING SYSTEM

The standard Linux shell provides powerful system-level command execution, but it has some limitations for student users.

*Drawbacks

❌ No student-specific utilities are available.
❌ Students need to use separate applications for academic-related tasks.
❌ Linux commands may be difficult for beginners to learn.
❌ There is no integration of student utilities with standard shell commands.
❌ The interface is not specifically designed for student requirements.
❌Students may need to remember multiple commands to perform different operations.

HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements

- **Processor**: Intel/AMD Processor
- **RAM**: 4 GB or above
- **Storage**: 10 GB free space
- **System**: Laptop/Desktop

Software Requirements

- **Operating System**: Ubuntu/Linux
- **Programming Language**: C
- **Compiler**: GCC
- **Code Editor**: VS Code
- **Version Control**: Git & GitHub

ALGORITHMS / OS CONCEPTS USED

Our project uses the following Operating System concepts:

1. Command Parsing

- Reads the command entered by the user.
- Separates the command and its arguments.
- Identifies the required operation.

2. fork() — Process Creation

- Creates a new child process.
- The child process is used to execute external commands.

3. exec() — Command Execution

- Executes the requested Linux command.
- Replaces the child process with the required program.

4. wait() — Process Synchronization

- The parent process waits for the child process to finish.
- Maintains proper execution order.

FUTURE ENHANCEMENTS

- **Student Database–**Store and manage student information.
- **Attendance Management–**Check and calculate student attendance.
- **Marks & CGPA Calculator–**Calculate marks, grades, and CGPA.
- **TimetableManagement–**View and manage class schedules.
- **Command History-**Store and access previously executed commands.

THANK YOU

presented by:

2510030094

2510030112

2510030351

2510030405